When Do Native Speech-Language Models Beat Cascades? A Latency–Accuracy Study of Constrained Telecom Retrieval

Esther Lee Yi Shan
Nanyang Technological University, Singapore

URECA Supervisor, Asst Prof, Dmitrii Ustiugov
Nanyang Technological University, Singapore

Collaborating Supervisors, Wenyan Chen, Hongrui Liu
Nanyang Technological University, Singapore

Abstract. Traditional speech-driven enterprise voice assistants rely heavily on cascade pipelines consisting of automatic speech recognition (ASR) modules coupled with standalone text-based large language models (LLMs). While highly modular, these architectures suffer from cascading error propagation and elevated latency overhead. The emergence of native omnimodal speech-language models (Omni-SLMs) offers a unified alternative by processing audio signals directly to generate text or speech. However, robust enterprise voice retrieval requires a challenging combination of joint speech understanding, exact entity preservation, retrieval and tool-use faithfulness, and low operational latency. This paper addresses whether native Omni-SLMs can effectively replace traditional cascaded ASR and LLM pipelines under these stringent constraints. We present an empirical evaluation comparing a high-performance cascade baseline (Whisper-large-v3 with Qwen2.5-7B) against two prominent native Omni-SLM architectures (Qwen3-Omni-30B and Nemotron-3-Nano-Omni-30B) on a constrained telecom customer retrieval and tool-use task. Evaluating over stratified real-world audio samples, we analyze the multi-dimensional performance frontier across stage-by-stage latency breakdowns and operational accuracy metrics. Our findings reveal that Omni-SLMs are not yet strictly superior to cascaded systems; rather, distinct architectural trade-offs emerge. The cascade pipeline remains the fastest option for real-time applications, whereas Qwen3-Omni yields outstanding vector alignment and database hit rates, and Nemotron-3-Nano-Omni provides superior hallucination suppression at the cost of a significant latency penalty. Ultimately, deployment decisions must be guided by whether execution speed, retrieval fidelity, or grounding precision dominates the enterprise requirements.

Keywords: Speech-Language Models, Cascade Pipelines, Vector Retrieval, Telecom Customer Assistant

1 INTRODUCTION

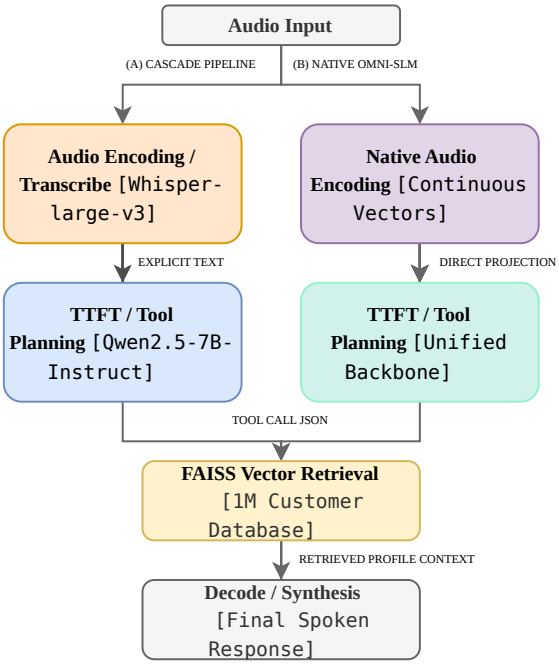

Figure 1: Architectural execution pathways of the modular Cascade baseline (A) versus the unified, native Omni-SLM pipeline (B) integrated with an intermediate FAISS customer database retrieval loop.

Enterprise voice assistants in domains like telecommunications are increasingly tasked with executing complex, data-grounded tasks in real-time. A prototypical workflow, contrasted across paradigms in Figure 1, involves listening to a customer query, iden-

tifying the intent to perform an account lookup, retrieving the corresponding user record from a massive vector database, and synthesizing a natural, context-grounded response.

Historically, this workflow has been addressed via a cascade baseline (Figure 1a). An automatic speech recognition (ASR) engine first transcribes the audio waveform to text. Subsequently, a text-based LLM parses the transcription, determines whether a tool execution is required, triggers database queries, and formats the final response. Although cascade pipelines permit individual module optimization, they introduce significant latency bottlenecks, particularly during transcription and second-stage model prefill phases, and suffer from transcription-error propagation [10].

The recent development of native, end-to-end Omnimodal Speech-Language Models (Omni-SLMs) has challenged this traditional pipeline paradigm. By aligning audio encoder representations directly into the embedding space of a large autoregressive language backbone, Omni-SLMs bypass explicit intermediate transcriptions (Figure 1b). While these architectures are theoretically optimized for fluid, lower-latency voice-to-voice interaction, their ability to execute structured tool-calling, maintain low hallucination scores on retrieved database entities, and minimize system latency remains largely unquantified in constrained enterprise tasks.

This study provides a rigorous, empirical evaluation of these competing paradigms. We evaluate:

1. A high-performance Cascade pipeline (Whisper-large-v3 coupled with Qwen2.5-7B-Instruct) [11].
2. Qwen3-Omni-30B-AWQ, representing quantized end-to-end speech models [13].
3. Nemotron-3-Nano-Omni-30B-BF16, representing unquantized, reasoning-capable mixture-of-experts (MoE) Omni-SLMs [6].

Evaluating across a stratified telecom customer database containing $1,000,005$ indexed records, we characterize the multi-dimensional performance frontier, mapping the exact trade-offs between latency budgets, tool execution precision, and factual grounding.

2 RELATED WORK

The architecture of speech-enabled conversational assistants has transitioned from strictly disjoint pipeline components toward end-to-end, multi-modal neural representations. This section reviews the literature surrounding modular cascaded systems, the emergence of native omnimodal architectures, and external database grounding paradigms.

2.1 CASCADED SPEECH-TO-TEXT-TO-REASONING PIPELINES

Traditional spoken dialogue and voice retrieval systems rely on a serial composition of modular interfaces [3]. Typically, an automatic speech recognition (ASR) frontend, such as the weak-supervision Whisper model family [11], first decodes raw acoustic waveforms into discrete text sequences. This textual transcript is subsequently ingested by a text-based large language model (LLM), such as Qwen2.5 [1], which performs semantic planning, intent classification, or tool execution.

While modularity allows developers to optimize or replace individual nodes independently, cascading architectures suffer from severe structural limits. First, they are highly vulnerable to transcription error propagation [3, 10]. Minor acoustic mismatches, out-of-vocabulary (OOV) terms, or phonetic errors introduced by the ASR module are passed to the downstream LLM as hard textual constraints. Because the LLM lacks access to the underlying acoustic confidence or prosodic cues, it often amplifies these initial transcription failures, leading to unrecoverable downstream reasoning errors [10]. Second, serial execution introduces a compounding latency bottleneck ($T_{\text{ASR}}+T_{\text{NLU}}+T_{\text{LLM}}$), presenting a severe barrier for real-time, interactive speech systems.

2.2 NATIVE SPEECH-LANGUAGE MODELS

To address the limitations of cascaded systems, recent research has explored native omnimodal speech-language models (Omni-SLMs) that align acoustic representations directly into a unified linguistic space [5, 13, 6]. Instead of converting speech into intermediate text, these models process raw audio inputs directly. For instance, the Qwen3-Omni architecture adopts a natively unified "Thinker-Talker" Mixture-of-Experts (MoE) design with an Audio Transformer (AuT) pre-trained front-end, decoupling cognitive reasoning from speech generation [13]. Other models utilize continuous projection layers (e.g., cross-attention or multi-layer perceptron adapters) to map frame-level acoustic features directly into the text embedding space of a transformer backbone [5].

By utilizing a unified autoregressive backbone for both speech comprehension and response synthesis, Omni-SLMs (such as Qwen3-Omni [13] and Nemotron-3-Nano-Omni [6]) eliminate the ASR-text bottleneck. This enables joint optimization of linguistic and paralinguistic features (such as tone, prosody, and emotion). However, serving these unified architectures introduces massive computational footprints. The dense, frame-level continuous audio features inflate the self-attention KV-cache compared to compact

text tokens, resulting in memory bandwidth bottlenecks during inference. To mitigate these serving costs, post-training compression techniques like Activation-aware Weight Quantization (AWQ) [9] have become vital for high-throughput deployment.

2.3 DATABASE GROUNDING AND TOOL RETRIEVAL

Retrieval-Augmented Generation (RAG) and tool call mechanisms are widely established for grounding text LLMs in external knowledge bases, preventing hallucination, and verifying facts [7]. Typically, an LLM parses a user query, generates a structured API or JSON tool call, and queries an external vector index, such as a Facebook AI Similarity Search (FAISS) database [7], to retrieve highly similar context profiles.

Adapting this tool-calling and database-grounding paradigm to native speech-language models represents an active research boundary. Because the model must translate continuous, noisy acoustic embeddings into highly structured, exact textual JSON arguments (like a phone number or a Customer ID), even tiny embedding shifts can corrupt the regex parsers. While text-only models process highly consolidated semantic concepts, speech models must preserve exact, dense sequential numeric details (such as telephone digits) across deep self-attention layers without suffering from semantic dispersion. Evaluating this tool call precision and the factual correctness of the resulting output remains a highly challenging, multi-dimensional task.

3 SYSTEM ARCHITECTURE & METHODOLOGY

To ensure a fair comparison, all models are evaluated within a unified customer database and tool-use framework.

3.1 VECTOR INFORMATION RETRIEVAL DATABASE

An enterprise telecom customer database was synthesized containing $1,000,005$ customer records: $1,000,000$ procedurally generated synthetic profiles, augmented with 5 manually inserted test profiles (`TEST_001`–`TEST_005`) used as ground-truth targets for the spoken lookup queries described in Section 4.2. Each record consists of a unique Customer ID, a randomized phone number, billing metrics, and a semantic profile. These records were encoded into 384-dimensional dense embeddings using the `all-MiniLM-L6-v2` sentence-transformer model [12], a lightweight 22M-parameter model chosen for fast embedding generation at the 1M-record scale, and L2-normalized prior to indexing using a fixed generation seed of 42. The normalized embeddings were indexed using a FAISS `IndexIVFFlat` structure (4,096 inverted-file clusters, $n_{\text{probe}} = 64$) with an L2-distance quantizer, trained directly on the full set of 1,000,005 embeddings before being added to the index. Because all vectors are unit-normalized, ranking by minimum squared L2 distance is mathematically equivalent to ranking by maximum inner product (cosine similarity), so retrieval behaves identically to an exact inner-product search while benefiting from IVF's sub-linear search cost at scale.

Formally, let the database be denoted as $\mathcal{D} = \{r_1, r_2, \ldots, r_N\}$ where $N = 1,000,005$. Each record r_i is mapped to a continuous dense vector $\mathbf{v}_i \in \mathbb{R}^D$ using an embedding function $E(\cdot)$ such that $\mathbf{v}_i = E(\text{flatten}(r_i))$. Given a natural language query q represented as a query vector $\mathbf{u}_q = E(q)$, the index search identifies the top K profiles by maximizing the inner product similarity metric:

$$\mathcal{R}_K(q) = \arg \max_{\mathcal{S} \subset \mathcal{D}, |\mathcal{S}|=K} \sum_{r \in \mathcal{S}} \langle \mathbf{u}_q, \mathbf{v}_r \rangle \tag{1}$$

where $\mathcal{R}_K(q)$ represents the set of retrieved candidate customer records. Equation 1 is expressed in inner-product form for clarity; in practice this is computed via the equivalent L2-minimization formulation native to the IVFFlat index described above.

3.2 TOOL EXECUTION & STRUCTURED FORMATTING

Models are given a strict system prompt containing operational rules. If a customer mentions a phone number or asks to retrieve account details, the model must output a structured JSON tool-call of the format:

```
{"function_call": {
  "name": "lookup_customer",
  "arguments": {"phone_number":
    "<number>"}}}
```

A regex parser intercepts this tool call, extracts the arguments, and queries the FAISS database index. The database returns the top matched profile and its associated FAISS distance. This retrieved database object is then fed back to the model's context window during the second-stage synthesis run, enabling it to answer the customer's query with factual accuracy.

We mathematically model this sequential execution. Let the generation of the response sequence $\mathbf{Y} = \{y_1, y_2, \ldots, y_T\}$ be conditioned on the system prompt template prefix $\mathbf{X}$ and the continuous acoustic representations $\mathbf{A}$ projected from the audio encoder:

$$P(\mathbf{Y}|\mathbf{X}, \mathbf{A}) = \prod_{t=1}^{T} P(y_t|y_{<t}, \mathbf{X}, \mathbf{A}) \tag{2}$$

When tool execution is triggered, the model generates a structured sequence representing the API call parameters $\theta = \text{RegexParse}(\mathbf{Y})$. The system intercepts θ, queries the FAISS database index to retrieve the semantic context $\mathcal{C} = \mathcal{R}_K(q)$, and appends $\mathcal{C}$ back to the model's context window. The final post-retrieval synthesis is formally expressed as:

$$P(\mathbf{Y}_{\text{final}}|\mathbf{X}, \mathbf{A}, \mathcal{C}) = \prod_{t=1}^{T'} P(y_t|y_{<t}, \mathbf{X}, \mathbf{A}, \mathcal{C}) \quad (3)$$

where T' is the length of the final generated sequence.

4 EXPERIMENTAL SETUP

4.1 MODEL SPECIFICATIONS

- **Cascade Baseline (Whisper-large-v3 + Qwen2.5-7B):** The audio waveform is transcribed using the original `openai-whisper` Python library (1.5B-parameter encoder-decoder transformer), not the Hugging Face transformers implementation, via `whisper.transcribe()` with default decoding settings (no explicit beam size, i.e. greedy decoding with the library's standard temperature fallback) and fp16 inference on CUDA. The text transcription is then wrapped in the system prompt template and passed to Qwen2.5-7B-Instruct served via vLLM [8], chosen for its PagedAttention memory management, which enables high-throughput serving of large models. All three models were served with `tensor_parallel_size=1` on a single GPU. Generation used greedy decoding (temperature 0.0, top-p 1.0, repetition penalty 1.05, max 512 tokens, fixed seed).
- **Qwen3-Omni-30B-AWQ:** A quantized variant of Alibaba's 30B parameter class Omni model. The model utilizes an aligned audio-visual projection layer and is compressed using Activation-aware Weight Quantization (AWQ) 4-bit precision to fit within standard GPU memory constraints. It is executed natively on vLLM [13]. The model was loaded with vLLM's `compressed-tensors` quantization backend, the loader format used for AWQ-packed weights.
- **Nemotron-3-Nano-Omni-30B-BF16:** Developed by NVIDIA [6], this model is a 31.6B parameter hybrid Mixture-of-Experts (MoE) architecture featuring a Mamba2-Transformer hybrid backbone. Operating with approximately 3.6 billion active parameters per token (including embeddings, or 3.2 billion excluding embeddings) during inference, though the complete 31.6 billion parameters must remain in memory, it features general-purpose agentic reasoning loops, with paralinguistic processing handled by its dedicated Parakeet audio encoder.

4.2 EVALUATION DATASET & HARDWARE

Table 1: Acoustic and Demographic Parameters of the $N = 500$ Evaluation Dataset

Acoustic Parameter	Value / Configuration
Sampling Frequency	$16.0\,\text{kHz}$ (general queries, resampled); native engine rate (lookup queries)
Resolution	16-bit Linear PCM
Channel Mode	Mono
Average Utterance Duration	$4.82\,\text{s}$
Spoken Format	Standard E.164 (International)
Stratification Ratio	20 / 80 (Stratified Interleaved)
TTS Engines	gTTS (general queries); pyttsx3 (lookup queries)

An evaluation dataset of $N = 500$ real-world audio samples was curated. To guarantee rigorous statistical validity, the files were stratified and interleaved into a perfect 20/80 mix: 100 lookup queries, each containing a spoken phone number drawn from 5 fixed test customers across 20 distinct conversational templates, and 400 general conversational queries. The 400 general queries were sourced from the **Banking77** dataset [4], using the held-out test split only (3,080 samples, of which 400 were sampled). Banking77 [4] is a benchmark of real customer service utterances spanning 77 fine-grained banking intents, originally introduced by Casanueva et al. (2020) for intent detection research. It was selected for this study because all 13,083 queries are general conversational questions containing no phone numbers, account identifiers, or structured lookup targets making every sample a ground-truth `requires_tool: False` negative in our binary tool-triggering evaluation. The 77 fine-grained intent labels from Banking77 are not used directly; instead, all Banking77 samples are uniformly assigned `requires_tool: False` and `intent: general`, collapsing the 77 classes into a single negative class for the purposes of tool-use precision and recall scoring.

Using the test split specifically avoids any risk of training data contamination for models that may have been pre-trained or fine-tuned on Banking77's training partition. The spoken audio clips were synthesized under clean, quiet, high-fidelity acoustic conditions using two local text-to-speech engines: gTTS [2] for general queries, and pyttsx3 for lookup queries. The pyttsx3 engine delegates to the platform-native TTS backend SAPI5 on Windows, NSSpeechSynthesizer on macOS, and espeak-ng on Linux and was configured at a speech rate of 155 words per minute. The choice of two separate engines reflects a deliberate acoustic diversity strategy: gTTS produces more naturalistic prosody resembling real customer speech, while pyttsx3 provides fully deterministic, offline synthesis that ensures exact reproducibility of the numeric phone-number utterances critical to the lookup evaluation. The detailed acoustic and demographic

configurations of this dataset are summarized in Table 1.

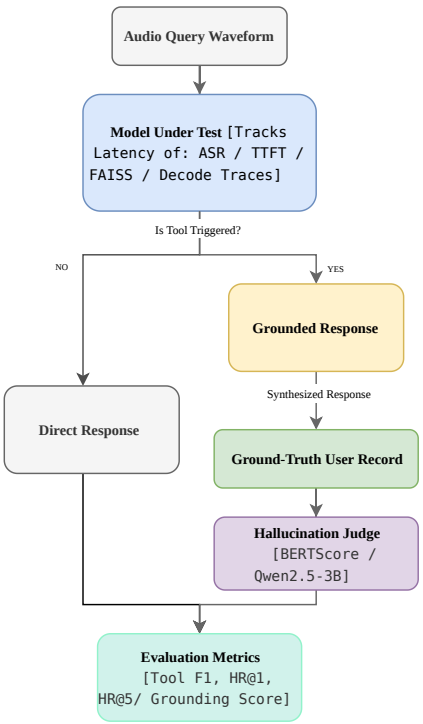

Figure 2: System design of the automated evaluation pipeline, tracing modular latency metrics and routing response outputs to the offline hallucination scoring judges.

The complete evaluation and benchmarking workflow, schematically mapped in Figure 2, isolates granular latency execution times while scoring synthesis responses against the active database index.

Experiments were conducted on an NVIDIA A100 GPU (80GB VRAM) paired with high-RAM system configurations. The vLLM V1 engine was configured under eager-execution modes with chunked prefill enabled, which splits long audio token sequences into smaller chunks during prefill to reduce peak memory usage and maximize parallel multimodal processing bandwidth [8].

4.3 EVALUATION METRICS

We characterize the system performance across five quantitative dimensions:

- **Tool-Use F1-Score:** Harmonic mean of precision and recall in determining whether a query requires a database lookup (True Positive) or direct conversational dialogue (True Negative).
- **FAISS Hit Rate (HR@K):** The proportion of successful lookups where the ground-truth user ID is retrieved within the top K database matches (for $K \in \{1, 5\}$).
- **Hallucination Score:** Evaluated using an automated semantic judge (BERTScore) comparing the synthesized response against the retrieved database profile to identify factual deviations in user metrics or billing details. Formally, let $x = \langle x_1, \ldots, x_M \rangle$ be the reference tokens of the retrieved database profile, and let $y = \langle y_1, \ldots, y_N \rangle$ be the generated response. Representing their contextualized token embeddings from a pre-trained transformer as $\mathbf{x}_i$ and $\mathbf{y}_j$, respectively, the recall (R_{BERT}) and precision (P_{BERT}) are computed using greedy cosine similarity matches:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{y_j \in y} \mathbf{x}_i^\top \mathbf{y}_j \tag{4}$$

$$P_{\text{BERT}} = \frac{1}{|y|} \sum_{y_j \in y} \max_{x_i \in x} \mathbf{x}_i^\top \mathbf{y}_j \tag{5}$$

The semantic hallucination score H is then calculated as:

$$H = 1 - 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{6}$$

While raw token-level BERTScore values theoretically span $[-1, 1]$, we apply baseline rescaling against the DeBERTa-xlarge-mnli corpus to project the metric into an empirical $[0, 1]$ interval, where 0 represents absolute factual alignment (perfect scaled BERTScore F-measure) and 1 represents complete semantic deviation.
- **End-to-End Latency (p_{50} and p_{95}):** Latency medians and tail-latency bounds measured from the millisecond the audio waveform is submitted until the final text character is decoded.
- **Per-Stage Latency Breakdown:** Isolating granular latency spent in ASR/audio-encoding, Time-to-First-Token (TTFT) planning, FAISS vector index search, tool execution, and autoregressive text decoding.

5 QUANTITATIVE EVALUATION

The quantitative results are summarized in Table 2 (Accuracy Profiles) and Table 3 (Latency Breakdowns).

5.1 ACCURACY, ALIGNMENT, AND HALLUCINATION SUPPRESSION

As shown in Table 2, the evaluated models display distinct accuracy trade-offs that are deeply rooted in their underlying architectures and parameter-serving formats:

- **Nemotron-3-Nano-Omni-30B-BF16** achieves the highest observed tool selection accuracy with an F_1-score of 0.985. Notably, it achieved a perfect precision rate (1.000 with 0 False Positives), showing an exceptional ability to suppress premature tool-calling on conversational queries. Furthermore, its fact-grounding was outstanding, yielding a near-zero mean hallucination score of 0.009 during response synthesis.

  Architectural Analysis: The model's superior precision and low hallucination rate are heavily driven by its hybrid Mamba2-Transformer Mixture-of-Experts (MoE) backbone operated at unquantized 16-bit precision (BF16). The 16-bit representation space preserves high-dimensional routing precision across its expert networks, while the Mamba2 layers provide exceptionally tight sequence-state tracking. This architectural synergy allows its MoE gating network to cleanly isolate reasoning tasks from conversational streams, eliminating "task cross-talk" and preventing conversational audio features from leaking into tool-calling embedding spaces. However, its lower FAISS Hit Rate (0.856) indicates a trade-off: without explicit numeric-preservation training or highly aligned projection constraints, raw acoustic representations of non-semantic digit sequences (such as phone numbers) can suffer from minor phonetic drift in unquantized MoE routing, leading to slightly mutated string parameters during database lookup.

- **Qwen3-Omni-30B-AWQ** achieved a perfect recall rate of 1.000 (0 False Negatives), meaning it never missed a database lookup when a phone number was present. This outstanding alignment carried over to its vector retrieval, securing a perfect FAISS Hit-Rate@1 of 1.000. However, it was slightly prone to over-triggering tool calls on general queries (17 False Positives), and showed a mean hallucination score of 0.135.

  Architectural Analysis: The perfect hit rate and recall stem from Qwen3-Omni's highly aligned Thinker–Talker MoE framework paired with Activation-aware Weight Quantization (AWQ). Because AWQ selectively protects the most prominent, high-magnitude activation channels (which typically represent literal features like numerical digit sequences), the model preserves the exact sequence of phone digits even under high compression. However, unlike the unquantized Nemotron, Qwen3-Omni-30B-AWQ suffers from the side-effects of 4-bit weight quantization. The aggressive compression compromises the precision of the MoE routing gates and shrinks the semantic attention manifold. This noise in the routing boundaries makes the model prone to "trigger-happy" tool calls (17 False Positives) when conversational audio features overlap with command templates, and results in coarser-grained text decoding that elevates the hallucination score (0.135) during non-protected metrics synthesis.

- The **Cascade Baseline** demonstrated highly robust, balanced performance across the board, securing an F_1-score of 0.922 and a FAISS Hit Rate@1 of 0.978, proving that standard pipeline baselines remain highly competitive.

  Architectural Analysis: The robustness of the cascade baseline lies in its discrete boundaries. Whisper-large-v3 acts as a powerful acoustic-to-text transduction module, converting continuous acoustic signals into hard text tokens before passing them to the Qwen2.5-7B LLM.This discretization acts as a semantic shield, protecting the text model from continuous auditory noise. However, standard cascade implementations typically do not propagate Whisper's token-level log-probabilities (acoustic confidence scores) to the downstream LLM, effectively forcing the model to treat the textual transcript as a hard constraint. If Whisper makes even a single phonetic error on a phone digit, the downstream LLM has no mechanism to recover from the mistake, leading to unrecoverable database lookup failures.

5.2 LATENCY BREAKDOWNS AND TAIL LATENCIES

Table 3 exposes massive latency variations across the three architectures.

- The **Cascade Baseline** is the fastest system, registering a mean E2E latency of $5.53\,\mathrm{s}$ and a median (p_{50}) of just $3.66\,\mathrm{s}$. This speed is driven by its modularity: Whisper transcribes speech in only $788\,\mathrm{ms}$, and the lightweight 7B Qwen text backbone decodes tokens in $2.14\,\mathrm{s}$.

- **Qwen3-Omni-30B-AWQ** achieves highly competitive, real-time-ready latency, with a mean E2E of $8.73\,\mathrm{s}$ and a median of $6.95\,\mathrm{s}$. This is a major achievement for a 30B parameter model, made possible by the 4-bit AWQ quantization and the unified representation layer, which reduces prefill planning to just $407\,\mathrm{ms}$.

- **Nemotron-3-Nano-Omni-30B-BF16** suffered from severe latency bottlenecks, registering an average E2E of $52.59\,\mathrm{s}$, and a p_{95} tail latency stretching to over $132.8\,\mathrm{s}$. This latency is heavily concentrated in the decoding stage ($31.58\,\mathrm{s}$), as its internal architecture executes massive, multi-step reasoning chains directly over the audio embedding representations before generating output text.

Table 2: Operational Accuracy, Vector Alignment, and Hallucination Suppression Profiles ($N = 500$ audio samples per model; see note below for accuracy-scoring denominators)

Model	Tool Prec.	Tool Recall	Tool F1	TP	FP	FN	HR@1	Halluc. Score
Cascade Baseline	**0.957**	0.890	**0.922**	89	**4**	11	0.978	**0.116**
Qwen3-Omni-30B-AWQ	0.850	**1.000**	0.919	**96**	17	**0**	**1.000**	0.135
Nemotron-3-Nano-Omni-BF16	1.000	0.970	0.985	97	0	3	0.856	0.009

Note: Accuracy denominators (TP+FP+FN+TN) reflect samples that completed tool-selection scoring: 500 / 496 / 502 for Cascade / Qwen3-Omni / Nemotron, respectively. Nemotron's substantially longer per-sample runtime produced 505 raw latency traces (Table 3), of which 502 completed full accuracy scoring; raw per-sample logs were not retained to isolate the exact cause of this 3-sample gap.

Table 3: Latency Performance Breakdown and Quantiled Tail Latencies (ms)

Model	Mean E2E	p50 E2E	p95 E2E	ASR / Enc	TTFT	FAISS	Tool Exec	Decoding
Cascade Baseline	**5,529**	**3,655**	**14,916**	**788**	**364**	1,047	1,065	**2,137**
Qwen3-Omni-30B-AWQ	8,732	6,948	20,381	915	407	**67**	**67**	4,056
Nemotron-3-Nano-Omni-BF16	52,593	39,287	132,815	4,290	2,502	1,098	1,116	31,577

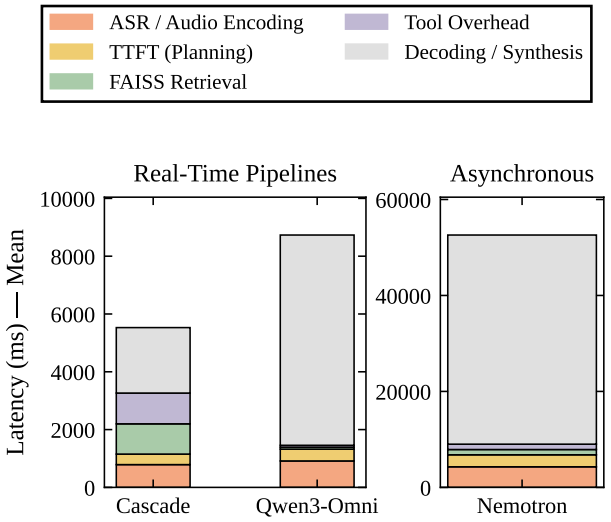

Figure 3: Latency breakdown by generation stage (p_{50} medians) showing the operational speed of Cascade Baseline compared to the reasoning-intensive cycles of Nemotron-3-Nano-Omni.

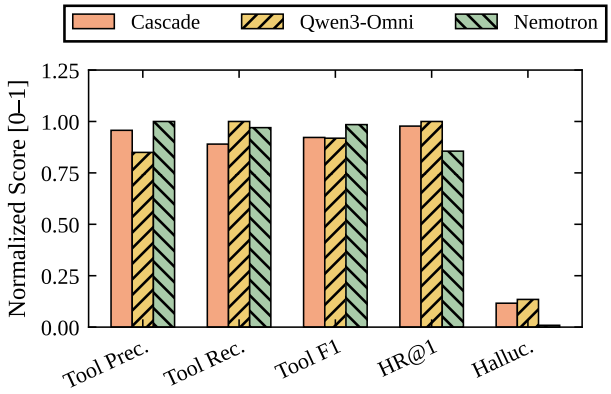

Figure 4: Operational metric comparisons showing the tool-use precision, recall, and factual grounding profiles across all three benchmark models.

6 DISCUSSION & LIMITATIONS

Our empirical findings reveal a distinct trade-off frontier that should guide enterprise architecture designs.

6.1 THE SPEED VS. PRECISION FRONTIER

When designing customer-facing voice interfaces, the absolute upper bound of acceptable latency is typically $3.0\,\mathrm{s}$ to $5.0\,\mathrm{s}$. This makes the **Cascade Baseline** the only viable option for interactive, turn-taking voice lines. However, the Cascade system's performance is bottlenecked by its higher hallucination rate (0.116), meaning it requires additional text-based guardrails to ensure database accuracy.

Conversely, **Nemotron-3-Nano-Omni** acts as an exceptional high-fidelity processor. Its F1-score of 0.985

and near-zero hallucination score (0.009) mean it almost never generates false user records or incorrect billing statements. While its $52\,\mathrm{s}$ latency makes it completely unusable for live telephone agents, it is highly suited for asynchronous, post-call auditing tasks (such as offline call-log verification and database matching).

To understand the architectural roots of this latency penalty, we must examine the memory bandwidth and cache constraints of unquantized Mixture-of-Experts (MoE) models served at full 16-bit precision, such as Nemotron-3-Nano-Omni [6]. During decoding, the model must dynamically route token embeddings through distinct expert pathways over deep autoregressive steps. More importantly, the continuous audio frames used in native Omni-SLMs require many more input tokens compared to compact text strings (e.g., a 5-second audio clip produces hundreds of frame-level embeddings). This massive sequence length drastically inflates the size of the self-attention KV-cache. Loading these extensive caches and MoE parameter matrices from GPU memory at every single autoregressive step creates a severe memory bandwidth bottleneck, driving the decoding stage to a highly intensive $31.58\,\mathrm{s}$ median time.

6.2 THE VALUE OF QUANTIZATION

Qwen3-Omni-30B-AWQ establishes a highly compelling compromise. By leveraging AWQ 4-bit quantization, it compresses a massive 30B model to achieve an E2E median of $6.95\,\mathrm{s}$, which is within reach of real-time systems, while securing a perfect Hit-Rate@1 (1.000). This proves that post-training quantization is vital to making end-to-end Omni models viable for high-throughput commercial serving.

AWQ [9] successfully retains this high accuracy by protecting the activation outliers of the projection and attention layers. In traditional uniform quantization, the massive scale variations in continuous multi-modal features lead to severe round-off errors. These errors destroy high-precision sequential details like numerical phone numbers, leading to catastrophic retrieval failures. By scaling the weight channels based on active channel observations, AWQ compresses the model size to 17 GB without losing critical numeric preservation, explaining why Qwen3-Omni-30B-AWQ maintains a perfect FAISS Hit-Rate@1 (1.000) while executing at sub-7-second speeds.

6.3 LIMITATIONS & FUTURE WORK

This study was conducted using text-only synthesis outputs from omnimodal backbones. A major next step is evaluating voice-to-voice models generating raw speech waveforms directly (voice-out). Furthermore, our evaluations were conducted in quiet, high-fidelity environments. Future work must analyze how environmental noise and varying packet loss rates degrade the audio embeddings of Omni models compared to standard ASR cascading denoisers.

We also note that the 100 lookup queries are drawn from only 5 unique test customer profiles, each queried under 20 distinct conversational templates; consequently, the reported HR@1/HR@5 and lookup-side hallucination figures reflect retrieval consistency over a small, fixed set of ground-truth records rather than generalization across a large pool of distinct phone numbers. Future work should evaluate retrieval fidelity across a substantially larger and more diverse set of ground-truth lookup targets.

Additionally, we acknowledge that while BERTScore serves as an automated semantic proxy, it possesses inherent structural limitations as a hallucination judge compared to human annotations or extensive multi-agent LLM arbitration ensembles.

The limitations of BERTScore [14] as a hallucination judge are particularly critical when evaluating numeric-heavy retrieval tasks. Because BERTScore relies on contextual cosine similarity, it measures high-level semantic alignment rather than exact entity matching. For example, if a model misrepresents a customer's phone number or billing amount by a single digit (e.g., matching "+12345678901" with "+12345678902"), the BERTScore contextual embeddings will remain almost identical, yielding an artificially low hallucination score. In reality, this minor semantic shift represents a catastrophic retrieval and security failure. Future work must develop specialized, entity-aware evaluation protocols that combine semantic similarity with rigorous, rule-based exact string matching to score multimodal databases accurately. Moreover, while Cascade pipelines can employ dedicated frontend ASR denoising and spectral subtraction filters to stabilize inputs, native Omni-SLM architectures remain highly vulnerable to out-of-domain acoustic distortions, presenting an open challenge for real-world deployment.

7 CONCLUSION

This paper presented a rigorous empirical comparison between traditional Cascade pipelines and state-of-the-art Omni-SLM architectures. We showed that Cascade baselines remain the fastest option for low-latency dialogue loops ($5.5\,\mathrm{s}$ mean E2E), whereas 30B Omni models provide substantial gains in retrieval alignment and hallucination suppression.

Ultimately, our findings suggest that enterprise architectures should shift from generic, single-model deployments to hybrid, task-specific pipelines: deploying Cascade systems for active customer-agent phone routing, AWQ-compressed Omni models for real-time

retrieval-heavy agents, and unquantized, reasoning-heavy Omni models for offline high-fidelity data auditing.

ACKNOWLEDGMENTS

I would like to acknowledge the support from Nanyang Technological University URECA Undergraduate Research Programme for this research project.

The author would also like to thank the URECA Supervisors and collaborating researchers for their invaluable guidance, technical support, and helpful discussions throughout the development and evaluation of this project.

REFERENCES

[1] Yang An, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[2] Pierre-Nick Bain et al. gTTS: Google text-to-speech. `https://gtts.readthedocs.io`, 2014. Accessed: 2025.

[3] Luisa Bentivogli, Mauro Cettolo, Marco Gaido, Alina Karakanta, Alberto Martinelli, Matteo Negri, and Marco Turchi. Cascade versus direct speech translation: Do the differences still make a difference? In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2873–2887, Online, August 2021. Association for Computational Linguistics.

[4] Iñigo Casanueva, Tadas Temcinas, Daniela Gerz, Matthew Henderson, and Ivan Vulic. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on NLP for ConvAI - ACL 2020*, mar 2020. Data available at https://github.com/PolyAI-LDN/task-specific-datasets.

[5] Yunfei Chu, Jin Xu, Qian Yang, Haojie Wei, Xipin Wei, Zhifang Guo, Yichong Leng, Yuanjun Lv, Jinzheng He, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen2-audio technical report. *arXiv preprint arXiv:2407.10759*, 2024.

[6] Amala Sanjay Deshmukh et al. Nemotron 3 nano omni: Efficient and open multimodal intelligence. *arXiv preprint arXiv:2604.24954*, 2026.

[7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, et al. Awq: Activation-aware weight quantization for llm compression and acceleration. In *Proceedings of the 7th MLSys Conference*, 2024.

[10] Yizhou Peng, Ziyang Ma, Changsong Liu, Yi-Wen Chao, Xie Chen, and Eng Siong Chng. Proactive for uncertainty: Cause-aware error diagnosis and interactive clarification for spoken dialogue systems. *arXiv preprint arXiv:2605.25404*, 2026.

[11] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, pages 28492–28518. PMLR, 2023.

[12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.

[13] Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, et al. Qwen3-omni technical report. *arXiv preprint arXiv:2509.17765*, 2025.

[14] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. In *Proceedings of the International Conference on Learning Representations*, 2020.